

Fintech Ecosystem Blueprint

Project: Fintech Ecosystem — Loan Origination, Digital Wallet, Real-Time Settlement

Goal: Build a production-grade, modular fintech ecosystem consisting of Loan Service, Wallet Service, and Settlement/Payment Gateway, integrated via event-driven architecture (Kafka), secured by API Gateway and OAuth2, deployed to Kubernetes with GitOps CI/CD. This project will be used as a case-study portfolio to demonstrate end-to-end fintech engineering skills suitable for international roles (e.g., Germany).

1. Executive Summary

This blueprint details a full-stack fintech ecosystem that simulates real-world requirements: loan origination & repayment, digital wallet with KYC & limits, and a settlement layer for real-time payments. It focuses on high reliability, data integrity, security, observability, and compliance basics (GDPR, PSD2 simulation, PCI awareness). The architecture is microservices-based, event-driven (Kafka), containerized (Docker + Kubernetes), and managed via GitOps (ArgoCD/Flux). CI/CD pipelines automate build, test, and deployment.

2. Business Requirements (High Level)

2.1 Core Features

Loan Service (LORS)

- User applies for loan with documents and basic info.
- Risk scoring (mock scoring engine) determines approval/denial.
- Approved loans are disbursed to user wallet or external bank account.
- Repayment schedules, installments, penalties for late payment.
- Audit trail for all actions.

Wallet Service

- User registration + KYC (mock external KYC provider).
- Top-up (deposit) to wallet via simulated bank transfer or external payment provider.
- Wallet balance management, transfer to other wallets, withdraw to bank.
- Limits engine (daily/monthly transaction limits, single-transaction limits).
- Fraud detection hooks (velocity rules, geo/ip anomalies).

Settlement / Payment Gateway

- Real-time payment routing between wallets and external banks.
- Ensure idempotency, atomicity of transfer operations.
- Ledger service to record settled transactions (immutable ledger / append-only events).
- DLQ and retry processing for failed settlements.

2.2 Non-Functional Requirements

- **Reliability:** 99.9% uptime target.
 - **Consistency:** ACID for money movements; eventually-consistent for reporting services.
 - **Security:** Encryption at rest & in transit; secure secrets management.
 - **Scalability:** Horizontal scaling using Kubernetes HPA and Kafka partitioning.
 - **Observability:** Metrics (Prometheus), tracing (OpenTelemetry), logs (ELK/EFK).
 - **Compliance:** GDPR data handling, ability to redact PII from logs.
 - **Testability:** Unit, integration, contract tests, chaos testing for resilience.
-

3. High-Level Architecture

```
[Clients] -> [API Gateway (Spring Cloud Gateway / Kong)] -> Auth Service (OAuth2) -> Routes to Microservices
```

Microservices:

- User Service (Auth, Profiles, KYC integration)
- Wallet Service (balances, ledger, limits)
- Loan Service (applications, scoring, schedules)
- Transaction Service (orchestrator for transfers)
- Settlement Service (external bank adapters)
- Notification Service (email/SMS/webhook)
- Reporting Service (analytics, exports)

Async integration: Kafka topics (events) -> Consumers

Data stores: PostgreSQL per-service (bounded context), Redis cache

Deployment: Docker -> Kubernetes (Helm) -> ArgoCD

CI/CD: GitHub Actions / GitLab CI

Monitoring: Prometheus + Grafana, Jaeger, ELK

3.1 Event Topics (examples)

- user.kyc.completed
 - wallet.topup.requested
 - wallet.topup.completed
 - transaction.initiated
 - transaction.settled
 - loan.application.submitted
 - loan.approval.changed
 - repayment.due
 - fraud.alert
-

4. Service Catalogue & Responsibilities

4.1 API Gateway

- Centralized entry point.
- JWT validation, rate limiting, per-client quotas.
- Request/response transformation (optional).

4.2 Auth & User Service

- OAuth2 Authorization Server (Keycloak or Spring Authorization Server).
- User profiles, roles (user, admin, operations).
- KYC orchestration: send docs to mock KYC provider, store verification result in encrypted store.

4.3 Wallet Service

- Core balance operations (credit/debit) with ledger entries.
- Idempotent APIs using idempotency keys.
- Daily/monthly limits engine.
- Exposes events: `wallet.*`.

4.4 Loan Service

- Workflow: application -> scoring -> approval -> disbursement -> repayment.
- Scoring engine: pluggable (start with rule-based, mock ML later).
- Use Saga pattern (choreography via events or orchestration via orchestration service) for disbursement to wallet.
- Schedule job: repayment due notices, auto-debit.

4.5 Transaction/Orchestrator Service

- Coordinates transfers between services where atomicity across services required.
- Offers transactional outbox pattern for sending events.

4.6 Settlement Service

- Settlement with external bank adapters.
- Handles incoming/outgoing settlements, idempotency, retries, DLQ.

4.7 Notification Service

- Send email/SMS/push/webhooks. Subscribe to events.

4.8 Reporting & Analytics

- Materialized views, OLAP store, export to Excel/PDF.

5. Data Model (Simplified)

5.1 Wallet Service

- `wallet` (id, user_id, currency, created_at)
- `balance` (wallet_id, amount, reserved_amount)
- `ledger_entry` (id, wallet_id, tx_id, type{CREDIT/DEBIT}, amount, balance_after, metadata, created_at)
- `limit` (wallet_id, type, period, amount)

5.2 Loan Service

- `loan_application` (id, user_id, amount, term_months, status, score, created_at)
- `loan_account` (id, loan_id, outstanding_balance, interest_rate)
- `repayment_schedule` (id, loan_id, due_date, principal, interest, status)

5.3 Transaction/Settlement

- `transaction` (id, source_wallet, dest_wallet, amount, currency, status, idempotency_key)
- `settlement_record` (id, transaction_id, external_ref, status, settled_at)

Encryption: PII fields stored encrypted using application KMS/Vault.

6. Integration Patterns & Data Consistency

- **Transactional Outbox:** Each service persists domain change and outbox event within same DB transaction. A separate dispatcher reads outbox table and publishes to Kafka.
 - **Saga Pattern:** For multi-service transactions (loan disbursement), use choreography (emitting/consuming events) or orchestration (a saga orchestrator service). Start with choreography for simplicity.
 - **Idempotency Keys:** All money-moving APIs accept idempotency keys stored in DB to prevent double processing.
 - **Optimistic Locking:** For balance updates, use optimistic locking (version column) or DB-level `SELECT FOR UPDATE` depending on RDBMS strategy. Consider serializable isolation for critical flows.
-

7. Security & Compliance

- **Authentication:** OAuth2 (Keycloak or Spring Authorization Server).
- **Authorization:** Role-based (RBAC) and attribute-based for sensitive operations.
- **Encryption:** HTTPS/TLS everywhere; encrypt PII at rest using HashiCorp Vault or cloud KMS.
- **Secrets Management:** Vault for DB credentials, JWT signing keys.
- **Audit Logging:** Immutable append-only audit logs; PII redaction in logs.
- **GDPR:** Right to be forgotten — implement data deletion/archival flows.
- **PCI Awareness:** Never store card PAN in plain text; tokenization or mock token.

8. Observability & Monitoring

- **Metrics:** Micrometer -> Prometheus. Key metrics: transaction throughput, error rates, latency (p95,p99), consumer lag (Kafka), DB query times.
 - **Tracing:** OpenTelemetry / Jaeger for distributed tracing across services.
 - **Logs:** Structured logs (JSON) -> ELK (Elasticsearch, Logstash/Filebeat, Kibana) or EFK. Mask PII.
 - **Dashboards & Alerts:** Grafana dashboards; alerting via Alertmanager (pagerduty/Slack).
-

9. Testing Strategy

- **Unit Tests:** JUnit + Mockito.
 - **Integration Tests:** Testcontainers for PostgreSQL & Kafka; run Spring context.
 - **Contract Tests:** Pact or Spring Cloud Contract for service contracts.
 - **End-to-End Tests:** Spin up environment (k8s in dev) and run scenario tests (loan application -> disbursement -> repayment).
 - **Chaos Testing:** Simulate Kafka broker down, DB slow queries, network partition to validate resilience.
-

10. CI/CD & Deployment

- **Git:** mono-repo or multi-repo per service. Recommend multi-repo for clearer boundaries, or mono-repo if pref for single pipeline.
 - **CI:** GitHub Actions/GitLab CI to run build, unit tests, integration tests (Testcontainers), security scans.
 - **Docker:** Build images, tag with semantic versioning, push to registry (DockerHub/ECR/GCR/Nexus).
 - **Helm Charts:** Per-service Helm charts, centralized umbrella chart for environments.
 - **GitOps:** ArgoCD watches `infra` Git repo (Helm values per environment) and syncs to k8s cluster.
 - **Environments:** dev (feature branches), staging (pre-prod), prod. Use separate namespaces and clusters if possible.
-

11. Deployment Topology & Infra

- **Kubernetes Cluster:** Managed (EKS/GKE/AKS) or self-managed RKE2.
 - **Namespaces:** `dev`, `staging`, `prod`.
 - **Ingress:** Nginx/Traefik + TLS via cert-manager.
 - **Service Mesh:** Optional: Istio/Linkerd for mTLS and advanced routing.
 - **Data store:** StatefulSets for Postgres (or managed RDS). Kafka as stateful cluster (Confluent or self-managed Strimzi).
 - **Storage:** PersistentVolumes for DB; object store (S3) for file uploads (KYC docs).
-

12. Roadmap & Milestones (12–20 weeks suggested)

Phase 0 (Week 0): Project setup

- Repos, CI templates, dev k8s cluster (local minikube / kind), Docker registry.

Phase 1 (Weeks 1–4): Core user & auth

- Build Auth/User service with OAuth2.
- Basic user registration, JWT, KYC mock.
- API Gateway routing.

Phase 2 (Weeks 4–8): Wallet core

- Wallet Service: ledger, balance, idempotency, top-up API.
- Integration with Notification.
- Unit & integration tests.

Phase 3 (Weeks 8–12): Transaction & Settlement

- Transaction service orchestration.
- Settlement adapters (mock bank). Kafka integration.
- DLQ & retry logic.

Phase 4 (Weeks 12–16): Loan Service

- Application -> scoring -> approval -> disbursement Saga.
- Repayment scheduler & auto-debit.

Phase 5 (Weeks 16–20): Observability & Hardening

- Prometheus/Grafana, tracing, logging.
- Security hardening, secrets management, helm charts.

Phase 6 (Weeks 20–24): E2E testing & polish

- Contract tests, load testing, docs, portfolio polish.

13. Learning Objectives & Where Weaknesses Show

Building this ecosystem will surface these weaknesses: - Distributed transactions & data consistency handling. - Idempotency & exactly-once processing patterns. - Kafka design: partitioning, consumer groups, offsets, schema registry. - Kubernetes production hardening & GitOps. - Security (OAuth2 flows, token lifecycle, KMS/Vault). - Observability across services (trace context propagation).

14. Tech Stack Recommendation

- Language: Java 17+
 - Framework: Spring Boot 3.x, Spring Security, Spring Data, Spring Cloud
 - Messaging: Apache Kafka + Confluent Schema Registry
 - DB: PostgreSQL + Redis
 - Auth: Keycloak or Spring Authorization Server
 - K8s: Kubernetes (managed or RKE2), Helm, ArgoCD
 - CI: GitHub Actions / GitLab CI
 - Monitoring: Prometheus, Grafana, Jaeger
 - Logs: EFK
 - Secrets: HashiCorp Vault or cloud KMS
-

15. Deliverables (Portfolio-ready)

- Public GitHub repos per service with README, architecture diagrams, and deployment scripts (Helm).
 - Playground dev cluster setup (kind/minikube) + local dev scripts.
 - Postman collection & API docs (OpenAPI/Swagger).
 - Demo video (5–10 mins) showing flows: loan application -> disbursement -> repayment.
 - Technical write-up: architecture decisions, trade-offs, resilience experiments.
-

16. Example API Endpoints (simplified)

Auth

- POST /auth/signup
- POST /auth/login

Wallet

- POST /wallets -> create wallet
- POST /wallets/{id}/topup -> topup with idempotency key
- POST /wallets/transfer -> transfer between wallets
- GET /wallets/{id}/balance

Loan

- POST /loans/apply
- GET /loans/{id}
- POST /loans/{id}/approve
- POST /loans/{id}/repay

Transactions

- `POST /transactions/settle`
 - `GET /transactions/{id}`
-

17. Runbooks (incident examples)

17.1 Consumer Lag spike

- Alert: Kafka consumer lag > threshold for X minutes.
- Runbook: Check consumer pod logs; check if consumer crashed; scale consumer replicas; check DB slowness causing backlog.

17.2 Double debit detected

- Alert: ledger balance negative or mismatched.
 - Runbook: Quarantine affected transactions; find idempotency keys; rollback via compensating transactions; notify compliance.
-

18. Cost & Resource Estimate (MVP)

- Dev infra: small k8s (3 nodes) + managed Postgres + Kafka cluster -> could run on cloud small instances (~\$200-\$500/mo for dev/staging).
 - Time: MVP (core flows) ~12-20 weeks with focused effort.
-

19. Next Steps — Immediate Action Plan (first 2 weeks)

1. Create repositories & git branching policy.
 2. Setup minimal dev k8s (kind/minikube) and Docker registry (local or remote).
 3. Implement Auth/User service + API Gateway skeleton.
 4. Implement CI pipeline template & basic Helm chart scaffold.
 5. Start Wallet service domain model & local dev integration with Kafka (Testcontainers).
-

20. Appendix: Learning Resources (by topic)

- **Spring & Java:** Official Spring Guides, Baeldung, Spring in Action, Vlad Mihalcea blog.
- **Kafka:** Confluent Kafka Fundamentals, "Designing Data-Intensive Applications" (Kleppmann).
- **Kubernetes:** Kubernetes Up & Running, Kelsey Hightower tutorials, Katacoda scenarios.
- **Security:** OWASP Top 10, Spring Security guides, Keycloak docs.
- **Observability:** OpenTelemetry docs, Prometheus docs, Grafana labs.
- **Distributed Systems:** Microservices Patterns (Chris Richardson), Martin Fowler.

Document generated as a complete blueprint. If you want, I can now create: - A detailed week-by-week checklist for Phase 1..6, - Helm chart skeleton and starter Spring Boot project templates, - Sequence diagrams and ER diagrams exported as PNGs, - CI pipeline YAML templates (GitHub Actions), - Sample code snippets for transactional outbox and idempotency.

Pilih salah satu tindakan selanjutnya, atau minta modifikasi blueprint ini sesuai preferensi lo.